



Advanced DSA Problems

Integration of Patterns & Multi-Concept Thinking

Java 8+



Session Structure

1. Multi-pattern integration
2. Graph + DP problems
3. Advanced Tree problems
4. Advanced String algorithms
5. Sliding window + hashing hybrids
6. Real interview-style problems
7. Optimization thinking



Problem 1: Trapping Rain Water

Concepts:

Two pointers + prefix/suffix max.

Time: $O(n)$

Space: $O(1)$

```
static int trap(int[] height) {
    int left = 0, right = height.length - 1;
    int leftMax = 0, rightMax = 0;
    int water = 0;

    while (left < right) {
        if (height[left] < height[right]) {
            if (height[left] >= leftMax)
                leftMax = height[left];
            else
                water += leftMax - height[left];
            left++;
        }
    }
}
```

```
else {  
    if (height[right] >= rightMax)  
        rightMax = height[right];  
    else  
        water += rightMax - height[right];  
    right--;  
}  
  
return water;  
}
```



Problem 2: Median of Two Sorted Arrays

Concept:

Binary search partition.

Time: $O(\log(\min(n, m)))$



Problem 3: Word Ladder (Graph + BFS)

Concept:

Graph traversal + shortest path.

Time: $O(N * M)$

Word Ladder Skeleton

```
static int ladderLength(String beginWord,
                        String endWord,
                        java.util.List<String> wordList) {

    java.util.Set<String> dict =
        new java.util.HashSet<>(wordList);

    java.util.Queue<String> queue =
        new java.util.LinkedList<>();

    queue.add(beginWord);
    int level = 1;

    while (!queue.isEmpty()) {
        int size = queue.size();

        for (int i = 0; i < size; i++) {
            String word = queue.poll();

            if (word.equals(endWord))
                return level;
        }
    }
}
```

```
char[] chars = word.toCharArray();

for (int j = 0; j < chars.length; j++) {
    char original = chars[j];

    for (char c = 'a'; c <= 'z'; c++) {
        chars[j] = c;
        String newWord =
            new String(chars);

        if (dict.contains(newWord)) {
            queue.add(newWord);
            dict.remove(newWord);
        }
    }

    chars[j] = original;
}

level++;
}

return 0;
}
```




Problem 4: Largest Rectangle in Histogram

Concept:

Stack + monotonic increasing stack.

Time: $O(n)$

Histogram Code

```
static int largestRectangle(int[] heights) {
    java.util.Stack<Integer> stack =
        new java.util.Stack<>();

    int maxArea = 0;

    for (int i = 0; i <= heights.length; i++) {
        int h = (i == heights.length) ? 0 :
            heights[i];

        while (!stack.isEmpty() &&
            h < heights[stack.peek()]) {

            int height =
                heights[stack.pop()];
            int width =
                stack.isEmpty() ? i :
                i - stack.peek() - 1;

            maxArea = Math.max(maxArea,
                height * width);
        }

        stack.push(i);
    }

    return maxArea;
}
```



Problem 5: Longest Palindromic Substring

Concept:

Expand around center.

Time: $O(n^2)$



Problem 6: Kth Largest Element

Concept:

Heap / QuickSelect.

Time:

$O(n)$ average.

THIRAVUGAL

QuickSelect Code

```
static int quickSelect(int[] nums,
    int left, int right, int k) {

    if (left == right)
        return nums[left];

    int pivot =
        partition(nums, left, right);

    if (k == pivot)
        return nums[k];
    else if (k < pivot)
        return quickSelect(nums,
            left, pivot - 1, k);
    else
        return quickSelect(nums,
            pivot + 1, right, k);
}
```



Problem 7: Alien Dictionary

Concept:

Topological sort + graph.

Time: $O(V + E)$

THIRAVUGAL

Problem 8: Maximum Subarray (Kadane)

Time: $O(n)$

```
static int maxSubArray(int[] nums) {  
    int max = nums[0];  
    int curr = nums[0];  
  
    for (int i = 1; i < nums.length; i++) {  
        curr = Math.max(nums[i],  
                        curr + nums[i]);  
        max = Math.max(max, curr);  
    }  
  
    return max;  
}
```



Problem 9: LRU Cache

Concept:

HashMap + Doubly LinkedList.

Time: $O(1)$ per operation.

THIRAVUGAL



Problem 10: Detect Cycle in Directed Graph

Concept:

DFS + recursion stack.

Time: $O(V + E)$



Multi-Concept Thinking

Advanced problems combine:

- Graph + DP
- Stack + Greedy
- Heap + Binary Search
- Sliding Window + HashMap



Optimization Strategies

1. Identify pattern
2. Reduce brute force
3. Analyze complexity
4. Optimize space
5. Consider edge cases



Interview Strategy

1. Clarify constraints
2. Start brute force
3. Improve step by step
4. Explain trade-offs
5. Discuss complexity



Complexity Awareness

Always answer:

Time complexity?

Space complexity?

Why optimal?

Edge cases?

THIRAVUGAL



Real World Applications

- Distributed systems load balancing
- Financial trading analytics
- Network routing optimization
- AI search & pruning
- Database query optimization



Common Mistakes

- Not identifying pattern
- Missing edge cases
- Incorrect boundary logic
- Overcomplicating simple problems
- Forgetting time-space tradeoff



Summary

Advanced DSA problems test:

- Pattern recognition
- Integration of concepts
- Optimization thinking
- Clean implementation
- Complexity explanation

Master:

- Sliding window + hashing
- Graph + BFS/DFS
- Stack monotonic pattern
- Heap & QuickSelect
- DP optimization